ORDANIS

PRIVATE WEALTH ADMINISTRATION

STANDARD OPERATING PROCEDURE · TECHNICAL

Building, Changing and Fixing ORDANIS

How the platform is put together, how changes reach production, and what to do when something goes wrong.

FOR	Platform administrators and developers
VERSION	October 29, 2026
CLASSIFICATION	Internal · Confidential
PUBLISHED BY	Ordanis OS LLC



Contents

01	Platform at a glance	What ORDANIS is made of and where each piece lives
02	The ground rules	The non-negotiables for anyone changing the platform
03	How a change ships	The exact path from an idea to production
04	Database changes	Migrations, grants and access rules
05	Server functions and scheduled jobs	What runs on the server, and when
06	People, roles and access	Who can see and do what, and where that is enforced
07	CHRIS: how it works under the hood	Centralized Household Records Integration Server, end to end
08	When something goes wrong	Symptom → cause → fix
09	Where everything else is written down	Runbooks, records and secrets

01 Platform at a glance

ORDANIS is a multi-enterprise client portal. A household gets a private place for its records, workflows, documents (CHRIS, the Centralized Household Records Integration Server) and an AI assistant. A firm can sit on top with view-only admin access and its own skin.

The stack

Layer	What it is	Where
Web app	React 18 single-page app built with Vite. One main file, `src/App.jsx` (about 16,000 lines); plan gating in `src/plans.js`; sign-up in `src/SignupFlow.jsx`.	Repo `pcm-realestate`
Database	Supabase Postgres with row-level security (RLS) on every household table.	Supabase projects (below)
Server functions	Supabase Edge Functions (Deno/TypeScript), one folder each in `supabase/functions/`.	Deployed per project
File storage	Supabase Storage. Private buckets: `documents` (CHRIS), `Videos` (plan preview videos), `brand`, `brand-documents`.	Supabase Storage
Scheduled jobs	`pg_cron` + `pg_net` inside Postgres call functions on a schedule, with a shared secret from Supabase Vault.	`cron.job`
Hosting	Vercel builds and serves the app on every push to `main`.	Project `ordanis-platform`
Payments	Stripe (checkout, subscriptions, webhook). Live-mode cutover is parked until launch.	`docs/RUNBOOK-stripe-live-cutover.md`
Email	Resend, sending from `no-reply@ordanisos.com` (set in `outbound_email_settings.fixed_from_email`).	Function secret `RESEND_API_KEY`
AI	Anthropic Claude via edge functions (assistant, document reading, filing suggestions, field extraction).	Function secret `ANTHROPIC_API_KEY`

Two environments

	Demo	Production
Git remote	`ordanis-demo` (ORDANISOS/titan-os, main)	`ordanis-production` (PCMFamilyOffice/pcm-realestate, main)
Supabase ref	`tkryueqzvgcigvxgjzsp`	`xreeruxuwtmjhvvxvjts`
App	Deploys on push	portal.ordanisos.com, deploys on push to main
Vercel	—	Project `ordanis-platform`, team ORDANIS OS

THE CLI IS LINKED TO PRODUCTION

The Supabase CLI in the repo folder points at production. Always pass `--project-ref` explicitly, and never run `supabase db push`.

Sibling apps (separate repos)

- **ELIOT**: the admins' voice console. Repo `ordanis-eliot` → eliot.ordanisos.com. Its back end lives here: `hub-command` and `public.audit_log`. Never add it to the portal.
- **MILO**: the marketing console (white theme). Repo `ordanis-milo` → milo.ordanisos.com. Back end here: `milo-command`, `milo-social`, `milo-speak`. MILO never sees households.
- **Marketing site**: `~/Documents/ordanis-site`, not in git; see its own CLAUDE.md.

02 The ground rules

These rules exist because each one was learned the hard way. They apply to people and to AI assistants alike.

1. **Nothing reaches production without Will's explicit approval, per step.** Approval for one step is not approval for the next.
2. **Demo first, always.** Every change is built and tested on demo, then queued as a production step.
3. **Never `supabase db push`. Never a bare `git push`.** Always name the remote: `git push ordanis-demo main` or `git push ordanis-production main`.
4. **Never move households or data from demo to production.**
5. **Never permanently delete a household.** Archiving is fine; `purge-family` is Will's call alone.
6. **Never enter, request or store passwords or keys**, and never put secrets in the repo, in chat, or in CLAUDE.md.
7. **Stage specific files only.** The working tree carries unrelated modified files (`supabase/.temp/*`, drift copies, `*.bak`).
8. **Keep security top of mind.** RLS on every table, least-privilege grants, no investment advice from the assistant.

> **BRAND AND WORDING**
>
> The product is ORDANIS (never Titan). Documents live in CHRIS, the Centralized Household Records Integration Server (always spelled out on first mention; never called a "vault"). Staff are "ORDANIS Expert", never "adviser" (a regulated word). Navy #0A2540, gold #C9A961, paper #F8F6F1. Warm, plain-language voice.

03 How a change ships

1. Build on demo

1. Make the change in the repo. For `src/App.jsx`, use targeted, asserted string replacements; never retype the file.
2. Syntax-check, then run the tests:

```
npx --yes esbuild --loader=jsx --log-level=error < src/App.jsx > /dev/null
npm test
```

3. If the change needs the database, apply the migration to **demo** (section 04) and record it.
4. If it needs a function, deploy it to **demo**:

```
supabase functions deploy <name> --project-ref tkryueqzvgcigvxgjzsp --use-api
```

5. Commit only the files you changed, then push to demo:

```
git add <files> && git commit -m "..." && git push ordanis-demo main
```

6. Test on demo, including a rolled-back SQL test for anything that touches access rules (section 06).

2. Promote to production (one approved step at a time)

1. **Database first** when the new app code depends on a new column or table; otherwise the live app breaks in the gap.
2. Deploy any function with `--project-ref xreeruxuwtmjhvvxvjts --use-api`, after saving the deployed source for rollback.
3. Will pushes the app from his terminal:

```
cd ~/Documents/GitHub/pcm-realestate && git push ordanis-production main
```

4. Confirm the build is live: `https://portal.ordanisos.com/version.json` shows the git short sha and build time.

> **IF A PUSH PRODUCES NO VERCEL BUILD**
>
> Open the Vercel project → Settings → Git. The repository link was once found disconnected.

04 Database changes

Writing a migration

- One file per change in `supabase/migrations/`, named `YYYYMMDDHHMMSS_short_name.sql`, with a header comment that says why, and a **Rollback** section.
- Re-read the file for any `update`, `delete` or `alter column` on existing tables before applying.
- **Every new table grants explicitly** (Supabase change of 2026-10-30): `service_role` select/insert/update/delete; `authenticated` only what the app needs (RLS still filters); `anon` nothing unless truly public. Never copy a template that grants select to anon.
- **RLS on every new table**, with the restrictive policies the household tables share where relevant (`mfa_required`, `private_household`, `household_perm_*`).
- **Security-definer functions**: pin `search_path`, then `revoke execute ... from public, anon; grant execute ... to authenticated` (or `service_role` only for jobs).

Applying it

1. Apply to demo, verify, record. Then, with approval, the same on production.
2. Production's migration history does **not** line up with repo file names, so apply **one file at a time** and record each:

```
insert into supabase_migrations.schema_migrations (version, name)
select '20261029120000','loan_secured_by_account'
where not exists (select 1 from supabase_migrations.schema_migrations where
version='20261029120000');
```

3. Trigger and storage-policy DDL can hang through the MCP tool. Wrap it in timeouts; if it still hangs, run it in the Supabase SQL editor:

```
set lock_timeout = '8s'; set statement_timeout = '60s';
```

Key tables

Table	Holds
`families`	One row per household: plan, Expert (`advisor_email`), firm (`enterprise_id`), Stripe ids, archive state.
`user_profiles`	Every login: `role` (admin, advisor = ORDANIS Expert, analyst, client, partner, enterprise_admin), `family_id`, `household_role` (primary/member) and the four family permissions.

`properties`, `portfolio_accounts`, `account_balances`, `loans`, `valuables`	The balance sheet. A property's mortgage and second mortgage live on the property; other debt lives in `loans`, optionally secured by a property, portfolio account or valuable.
`cash_flow_events`, `cash_flow_payment_log`	Income and bills, recurrence, who pays (`pcm_responsible`), payments per period.
`documents`, `document_folders`, `document_downloads`	CHRIS (section 07).
`tasks`, `notes`, `obligations`, `workflow_*`	Work: tasks, notes, and workflows (stored in `obligations`, shown everywhere as "workflows") with the playbooks that carry them.
`plan_features`, `plan_change_requests`	Plan prices and gates (the single source for app and database); requests to change plan.
`household_activity`	Client Activity: every add/change/remove a family login makes (field names only).
`expert_analysts`, `household_onboarding_specialists`, `family_partners`	Staffing: Analyst per Expert, 30-day Onboarding Specialists, Business Partner seats.
`audit_log`	Administrative actions (also ELIOT's log).
`brand_profiles`, `outbound_email_settings`	Skin per host/firm; the sending address.

05 Server functions and scheduled jobs

Deploying a function

```
supabase functions deploy <name> --project-ref xreeruxuwtmjhvvxvjts --use-api
```

- Deploy only the functions named in the approved step.
- Before replacing a function, read production's deployed source, diff it with the repo file, and save the old source for rollback.
- JWT verification is **off** (deploy with `--no-verify-jwt`) only for functions called by Stripe or by cron: `stripe-webhook`, `public-signup`, `send-task-reminders`, `onboarding-notices`, `milo-social`. Cron functions check the `x-cron-secret` header against `CRON_SECRET` and fail closed.
- Pin `npm:@supabase/supabase-js@2.49.4` in new functions.

The main functions

Function	Job
`family-ai-assistant`	The household assistant ("Ask <name>"). Answers from one family's snapshot and document text; refuses investment advice (regex pre-filter + model rules).
`extract-document-text`	Reads scanned PDFs and photos (OCR) so CHRIS can search them.
`suggest-document-folder`	CHRIS's filing brain: proposes a name, summary, folder and record link.
`extract-statement-fields`, `extract-property-fields`	Propose balances or property details from a statement for a person to confirm.
`send-task-reminders`	Daily: one summary email per ORDANIS Expert (task reminders, bills they pay, Client Activity) plus family reminders on Basic/Core.
`onboarding-notices`	Daily: day-30 notice when an Onboarding Specialist's access is ending.
`plan-request-notify`	Emails info@ordanisos.com when a household asks to change plan.
`change-subscription-plan`, `create-checkout-session`, `cancel-subscription`, `stripe-webhook`	Billing.

`invite-client`, `public-signup`, `firm-join`, `manage-business-partner`	Getting people in.
`draft-workflow-step`, `send-workflow-step`, `purchase-workflow`, `request-expert-assist`	Workflows; nothing sends without a named human approval.
`archive-family`, `purge-family`	Archive (reversible) and purge (permanent: Will only).

Scheduled jobs (production, all active since go-live)

Job	Schedule (UTC)	Calls
send-task-reminders	13:00 daily	Expert daily summary + family reminders
onboarding-notices	13:10 daily	Day-30 specialist notices
run-scheduled-prompts	hourly	Scheduled assistant reports
eliot-briefings	every 5 min	ELIOT (`hub-command`)
milo-hourly	:07 hourly	MILO
workflow-overage-monthly	03:15 on the 1st	Workflow overage billing
enterprise-monthly-snapshot	06:15 on the 1st	Firm snapshot

```
-- what ran, and how it went
select jobid, status, start_time, return_message from cron.job_run_details order by start_time
desc limit 20;
-- the function's own reply to a pg_net call
select status_code, content from net._http_response order by id desc limit 5;
-- pause / resume a job
select cron.alter_job(<jobid>, active := false);
```

06 People, roles and access

The screen hides what a person can't use, but the **database** is the enforcement. Every rule below holds even if someone calls the API directly.

Role	Sees	Can change
Admin	Everything	Everything (some actions need two-factor, aal2)
ORDANIS Expert (`advisor`)	Households where `families.advisor_email` is theirs	Those households
Analyst	Their Expert's households (via `expert_analysts`)	Those households
Onboarding Specialist	Households they're assigned to, for 30 days (extendable)	Those households
Family (`client`)	Their own household	Primary: everything. Members: what the primary ticked (edit, upload, download, manage users)
Business Partner	Households that granted them a seat	Nothing except uploading documents
Firm administrator	Their firm's households	Nothing (view only)

- One role per login. Admins can also fill the Expert, Analyst and Onboarding Specialist seats.
- Every plan (Basic, Core, Premier) edits the same household records. Changes by family logins land in `household_activity` and on the Expert's daily summary.
- Edit Family's plan picker is locked for Stripe-paid households: plan changes go through Plan requests or Billing so Stripe moves too.

Testing an access rule safely

Pretend to be a user inside a transaction that always rolls back:

```
do $$ declare n int; begin
  perform set_config('request.jwt.claims','{"sub":"<user-
uuid>","role":"authenticated","aal":"aal2"}', true);
  set local role authenticated;
  select count(*) into n from public.household_activity;
  reset role;
  raise exception 'ROLLBACK sees %', n;   -- nothing is kept
end $$;
```

07 CHRIS: how it works under the hood

CHRIS in one paragraph

CHRIS, the Centralized Household Records Integration Server, is not a folder of files. Every document is stored privately, read by AI (including scans and phone photos), named and summarised, filed into the right folder, searchable by what is *inside* it, and pinned to the exact figure it supports: a property's tax bill, a valuable's policy, a loan's statement, an account's balance. The assistant answers from it. Every download is logged.

The pieces

Piece	How
Storage	Private bucket `documents`, path under the household's own folder (which the storage policies check). Files are only ever opened through short-lived signed URLs.
Rows	`documents`: name, category (folder), description (AI summary), `extracted_text`, links (`property_id` + `property_section`, `valuable_id`, `loan_id`, `account_id`), `expiry_date`, `private_owner_id`.
Reading	Digital PDFs are read in the browser; scans and photos go to `extract-document-text`. A file that can't be read still uploads.
Filing	`suggest-document-folder` proposes name, summary, folder (or a new one), and a record link with a confidence. Nothing is filed until a person taps File.
Folders	Eight built in (General, Tax, Legal, Insurance, Investment, Real Estate, Estate Planning, Other) plus each household's own (`document_folders`).
Replacing	Linking a new document to a spot unlinks the old one, never deletes it. Search marks replaced documents plainly.
Private folder	A family login's own files, hidden from everyone else by restrictive policies, and never read by AI.
Audit	`document_downloads` records who opened what, and when. A firm administrator's open is recorded before the file is served.

Permissions	Family members need "Upload" to add files and "Download" to open them (storage policies `household_perm_files_*`).

When CHRIS misbehaves

- **Search can't find a document:** its `extracted_text` is empty. Staff press ✦ **Scan** on the row (or the bulk Scan button), or check the function logs for `extract-document-text`.
- **No filing suggestion:** the text was too short or the AI call failed. The person can still choose a folder by hand.
- **"Permission denied" on open or upload:** check the member's Download/Upload permission and the storage path (it must start with the household's id).

08 When something goes wrong

Symptom	Usual cause	Fix
"Edge Function returned a non-2xx status code"	The function refused for a reason it explained, but the app hid the body.	Read it: `await error.context.json()`. In the database, `net._http_response` or the function logs in Supabase.
"permission denied for table …"	A table created after 2026-10-30 without explicit grants.	Add the grants (section 04). Check RLS policies too.
Push made no new build	Vercel's Git link disconnected.	Vercel → Settings → Git → reconnect; push again.
Site shows old version	Build still running, or a cached tab.	Check `/version.json`. Open tabs are told when a newer build is live.
Migration hangs	Trigger or storage-policy DDL waiting on a lock.	Timeouts (section 04); else run in the SQL editor.
No emails arriving	Resend error, sending address unset, or demo override.	Function logs; `outbound_email_settings.fixed_from_email`; `DEMO_EMAIL_OVERRIDE` on demo.
Daily summary didn't go	Cron job paused or the function failed.	`cron.job_run_details`, then `net._http_response`. Items not sent are retried the next day.
Plan shows one thing, Stripe bills another	Plan changed by hand on a Stripe-paid household.	Approve through Plan requests (moves Stripe), or record with "no billing" deliberately.
Assistant refuses a question	The investment-advice guardrail, by design.	Not a bug for client/partner users. `brand_profiles.ai_investment_policy` only changes how much factual detail is shown.
A family member can't edit or upload	Their household permissions.	The primary user (or an admin) ticks the permission in the Household tab.

Rolling back

- **App:** in Vercel, promote the previous production deployment (instant); then fix forward with a new commit. Or `git revert` the commit and push.
- **Function:** redeploy the saved source you kept before replacing it.

- **Database:** run the migration's Rollback section (one file at a time), then remove its row from `supabase_migrations.schema_migrations`.
- **Data:** never purge. Archive a household instead; ask Will about anything permanent.

09 Where everything else is written down

Document	Covers
`CLAUDE.md` (repo root)	Standing rules, environments, current state, go-live notes, tabled launch items.
`docs/RUNBOOK-stripe-live-cutover.md`	Turning on live payments.
`docs/RUNBOOK-MILO.md`, `docs/RUNBOOK-ELIOT-EA.md`	The sibling consoles.
`docs/HANDOFF-*.md`	Plans, spend and vendors.
`supabase/migrations/`	Every database change, each with its reason and rollback.
Claude Doc "ORDANIS Production Release Plan"	Release steps and their status.

Secrets (names only, never values)

Edge functions: `RESEND_API_KEY`, `ANTHROPIC_API_KEY`, `STRIPE_SECRET_KEY`, `CRON_SECRET`, `PLAN_REQUEST_NOTIFY_TO` (optional), `DEMO_EMAIL_OVERRIDE` (demo only). Supabase Vault: `cron_secret` (must match `CRON_SECRET`). Vercel: `VITE_BRAND_RUNTIME=1` on production. Secrets are set in the Supabase and Vercel dashboards by Will; they never go in the repo, in chat, or in documents.

Tabled until the official launch

1. Stripe end-to-end test and live-mode gates.
2. Expert-assist test on a card-paid Core household (refund the $25).
3. Reopen sign-up: set `SIGNUP_PAUSED_URL` to "" in `src/App.jsx`.